

[Help](#) [Docs](#) [Sponsors](#) [Log in](#) [Register](#)

mrmustard 0.7.3

Differentiable quantum Gaussian circuits

```
pip install mrmustard
```

[Description](#) [Download files](#) [Release history](#)

License [Apache-2.0](#) [Actions Status](#)

python [3.9](#) | [3.10](#) | [3.11](#)

Mr Mustard is a differentiable simulator with a sophisticated built-in optimizer, that operates seamlessly across phase space and Fock space. It is built on top of an agnostic autodiff interface, to allow for plug-and-play backends (`numpy` by default).

Mr Mustard supports:

- Phase space representation of Gaussian states and Gaussian channels on an arbitrary number of modes
- Exact Fock representation of any Gaussian circuit and any Gaussian state up to an arbitrary cutoff
- Riemannian optimization on the symplectic group (for Gaussian transformations) and on the unitary group (for interferometers)

Key dates

[PyPI data >](#)

Released: Mar 28, 2024

LATEST RELEASE

2 maintainers

[PyPI data >](#)

XanaduAI



ziofil

Credits

Author: [Xanadu](#)

License

- Adam optimizer for euclidean parameters
- Single-mode gates (parallelizable):
 - squeezing, displacement, phase rotation, attenuator, amplifier, additive noise, phase noise
- Two-mode gates:
 - beam splitter, Mach-Zehnder interferometer, two-mode squeezing, CX, CZ, CPHASE
- N-mode gates (with dedicated Riemannian optimization):
 - Interferometer (unitary), RealInterferometer (orthogonal), Gaussian transformation (symplectic)
- Single-mode states (parallelizable):
 - Vacuum, Coherent, SqueezedVacuum, Thermal, Fock
- Two-mode states:
 - TMSV (two-mode squeezed vacuum)
- N-mode states:
 - Gaussian
- Photon number moments and entropic measures
- PNR detectors and Threshold detectors with trainable quantum efficiency and dark counts
- Homodyne, Heterodyne and Generaldyne measurements
- Composable circuits
- Plug-and-play backends (`numpy` as default)
- An abstraction layer `XPTensor` for seamless symplectic algebra (experimental)

Increased numerical stability using Julia [optional]

Converting phase space objects to Fock space can be numerically unstable due to accumulating floating point errors. To resolve this, the conversion can be

Apache Software License (Apache-2.0)

Requires

Python <3.12, >=3.9

Classifiers

Development Status

- [4 - Beta](#)

Environment

- [Console](#)

Intended Audience

- [Science/Research](#)

License

- [OSI Approved :: Apache Software License](#)

Natural Language

- [English](#)

Operating System

- [MacOS :: MacOS X](#)
- [Microsoft :: Windows](#)
- [POSIX](#)
- [POSIX :: Linux](#)

Programming Language

- [Python](#)
- [Python :: 3](#)
- [Python :: 3 :: Only](#)
- [Python :: 3.9](#)
- [Python :: 3.10](#)
- [Python :: 3.11](#)

Topic

- [Scientific/Engineering :: Physics](#)

[Report project as malware](#)

performed with extended-precision arithmetic. To use this feature, an installation of [Julia](#) is required (version 1.9.3 recommended).

- When installing MrMustard via devcontainer, Julia and its required packages are automatically installed.
- When installing MrMustard via `poetry install` or `pip install`, the required Julia packages have to be installed manually as follows:

```
julia --project="julia_pkg" -e "using Pkg; Pkg.add(Pkg.PackageSpec(; url="https://github.com/quantumlib/
```

- When installing MrMustard via the `MakeFile`, the required Julia packages are automatically installed *only* if Julia was previously installed by the user.

The lab module

The lab module contains things you'd find in a lab: states, transformations, measurements, circuits. States can be used at the beginning of a circuit as well as at the end, in which case a state is interpreted as a measurement (a projection onto that state). Transformations are usually parametrized and map states to states. The action on states is differentiable with respect to the state and to the gate parameters.

1. States and Gates

Here are a few examples of states and gates:

```
import numpy as np
from mrmustard.lab import *

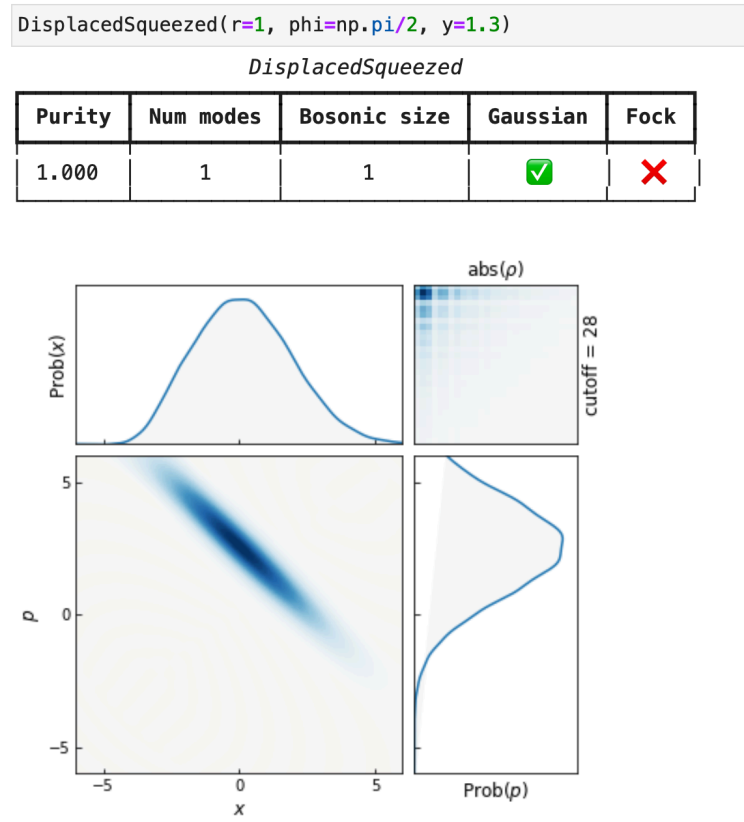
vac = Vacuum(num_modes=2)          # 2-mode vacuum state
coh = Coherent(x=0.1, y=-0.4)      # coherent state
```

```
sq = SqueezedVacuum(r=0.5)      # squeezed
g  = Gaussian(num_modes=2)      # 2-mode Ga
fock4 = Fock(4)                  # fock stat

D = Dgate(x=1.0, y=-0.4)        # Displace
S = Sgate(r=0.5)                # Squeezer
R = Rgate(angle=0.3)            # Phase ro
A = Amplifier(gain=2.0)          # noisy an
L = Attenuator(0.5)              # pure los
N = AdditiveNoise(noise=0.1)    # additive

BS = BSgate(theta=np.pi/4)      # 50/50
S2 = S2gate(r=0.5)              # two-mo
MZ = MZgate(phi_a=0.3, phi_b=0.1) # Mach-Z
I = Interferometer(8)           # 8-mode
L = Attenuator(0.5)              # pure T
A = Amplifier(gain=2.0, nbar=1.0) # noisy
```

The `repr` of single-mode states shows the Wigner function:

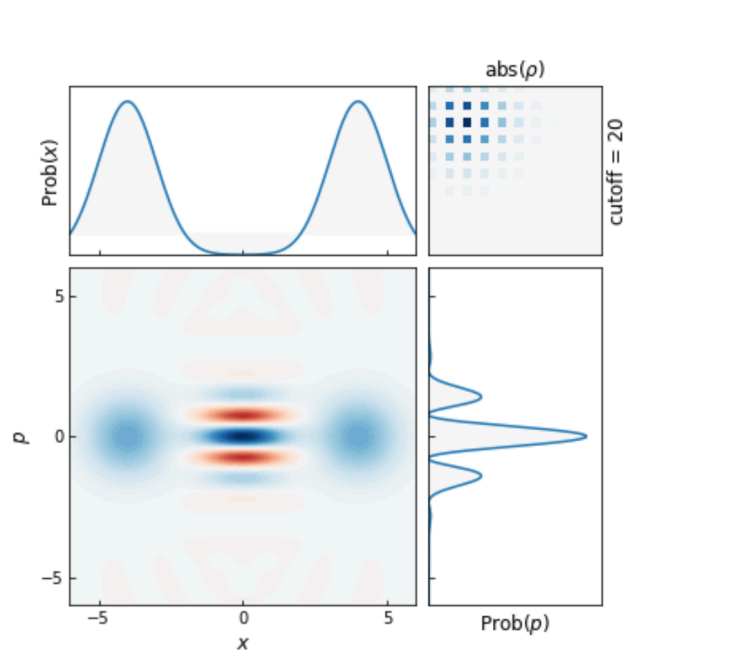


```
cat_amps = Coherent(2.0).ket([20]) + Coherent(2.0).ket([20])
cat_amps = cat_amps / np.linalg.norm(cat_amps)
```

```
cat = State(ket=cat_amps)
cat
```

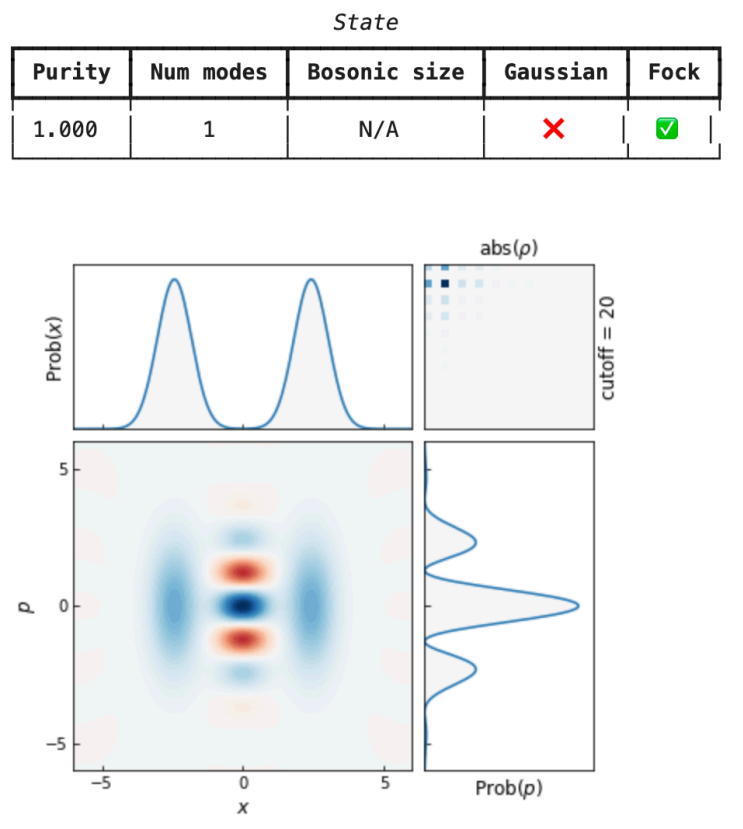
State

Purity	Num modes	Bosonic size	Gaussian	Fock
1.000	1	N/A	✗	✓



States (even those in Fock representation) are always compatible with gates:

```
cat >> Sgate(0.5) # squeezed cat
```



2. Gates and the right shift operator >>

Applying gates to states looks natural, thanks to python's right-shift operator >>:

```
displaced_squeezed = Vacuum(1) >> Sgate(r=0.1)
```

If you want to apply a gate to specific modes, use the `getitem` format. Here are a few examples:

```
D = Dgate(y=-0.4)
S = Sgate(r=0.1, phi=0.5)
state = Vacuum(2) >> D[1] >> S[0] # displaced and squeezed mode 0

BS = BSgate(theta=1.1)
state = Vacuum(3) >> BS[0,2] # applying a beam splitter to modes 0 and 2
state = Vacuum(4) >> S[0,1,2] # applying the squeeze gate to modes 0, 1, and 2
```

3. Circuit

When chaining just gates with the right-shift `>>` operator, we create a circuit:

```
X8 = Sgate(r=[1.0] * 4) >> Interferometer(4)
output = Vacuum(4) >> X8

# lossy X8
noise = lambda: np.random.uniform(size=4)
X8_noisy = (Sgate(r=0.9 + 0.1*noise(), phi=0)
            >> Attenuator(0.89 + 0.01*noise())
            >> Interferometer(4)
            >> Attenuator(0.95 + 0.01*noise())
            )

# 2-mode Bloch Messiah decomposition
bloch_messiah = Sgate(r=[0.1,0.2]) >> BSgate
my_state = Vacuum(2) >> bloch_messiah
```

4. Measurements

In order to perform a measurement, we use the left-shift operator, e.g. `coh << sq` (think of the left-shift on a state as "closing" the circuit).

```
leftover = Vacuum(4) >> X8 << SqueezedVacuum
```

Transformations can also be applied in the dual sense by using the left-shift operator `<<`:

```
Attenuator(0.5) << Coherent(0.1, 0.2) == Coherent
```

This has the advantage of modelling lossy detectors without applying the loss channel to the state going into the detector, which can be overall faster e.g. if the state is kept pure by doing so.

5. Detectors

There are two types of detectors in Mr Mustard. Fock detectors (PNRDetector and ThresholdDetector) and Gaussian detectors (Homodyne, Heterodyne, Generaldyne).

The PNR and Threshold detectors return an array of unnormalized measurement results, meaning that the elements of the array are the density matrices of the leftover systems, conditioned on the outcomes:

```
results = Gaussian(2) << PNRDetector(efficie
results[0] # unnormalized dm of mode 1 conc
results[1] # unnormalized dm of mode 1 conc
results[2] # unnormalized dm of mode 1 conc
# etc...
```

The trace of the leftover density matrices will yield the success probability. If multiple modes are measured then there is a corresponding number of indices:

```
results = Gaussian(3) << PNRDetector(efficie
results[2,3] # unnormalized dm of mode 2 co
# etc...
```

Set a lower `settings.PNR_INTERNAL_CUTOFF` (default 50) to speed-up computations of the PNR output.

6. Comparison operator `==`

States support the comparison operator:

```
>>> bunched = (Coherent(1.0) & Coherent(1.0))
>>> bunched.get_modes(1) == Coherent(np.sqrt(2))
True
```

As well as transformations (gates and circuits):


```
>>> Dgate(np.sqrt(2)) >> Attenuator(0.5) ==
True
```

7. State operations and properties

States can be joined using the `&` (and) operator:

```
Coherent(x=1.0, y=1.0) & Coherent(x=2.0, y=2.0)

s = SqueezedVacuum(r=1.0)
s4 = s & s & s & s    # four squeezed states
```

Subsystems can be accessed via `get_modes`:

```
joint = Coherent(x=1.0, y=1.0) & Coherent(x=2.0, y=2.0)
joint.get_modes(0) # first mode
joint.get_modes(1) # second mode

swapped = joint.get_modes([1,0])
```

8. Fock representation

The Fock representation of a State is obtained via `.ket(cutoffs)` or `.dm(cutoffs)`. For circuits and gates (transformations in general) it's `.U(cutoffs)` or `.choi(cutoffs)`, if available. The Fock representation is exact and it doesn't break differentiability. This means that one can define cost functions on the Fock representation and backpropagate back to the phase space representation.

```
# Fock representation of a coherent state
Coherent(0.5).ket(cutoffs=[5]) # ket
Coherent(0.5).dm(cutoffs=[5]) # density matrix
```

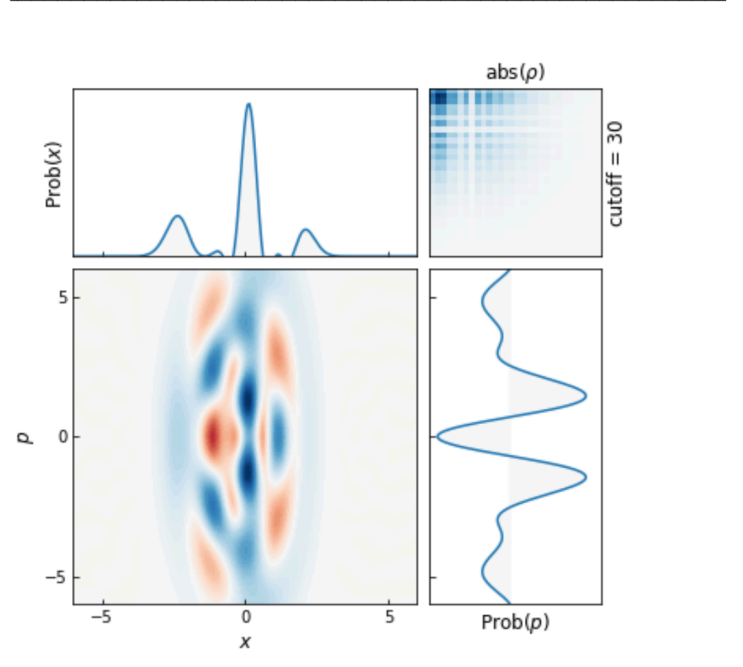
```
Dgate(x=1.0).U(cutoffs=[15]) # truncated ur
Dgate(x=1.0).choi(cutoffs=[15]) # truncated
```

States can be initialized in Fock representation and used as any other state:

```
my_amplitudes = np.array([0.5, 0.25, -0.5, 0.1])
my_state = State(ket=my_amplitudes)
my_state >> Sgate(r=0.5) # just works
```

State

Purity	Num modes	Bosonic size	Gaussian	Fock
1.000	1	N/A	✗	✓



Alternatively,

```
my_amplitudes = np.array([0.5, 0.25, -0.5, 0.1])
my_state = State(ket=my_amplitudes)
my_state._cutoffs = [42] # force the cutoff
my_state >> Sgate(r=0.5) # works too
```

The physics module

The physics module contains a growing number of functions that we can apply to states directly. These are made out of the functions that operate on the *representation* of the state:

- If the state is in Gaussian representation, then internally the physics functions utilize the [physics.gaussian](#) module.
- If the state is in Fock representation, then internally the physics functions utilize the [physics.fock](#) module.

The math module

The math module is the backbone of Mr Mustard. Mr Mustard comes with a plug-and-play backends through a math interface. You can use it as a drop-in replacement for tensorflow or numpy and your code will be plug-and-play too!

Here's an example where the `numpy` backend is used.

```
import mrmustard.math as math

math.cos(0.1)  # numpy
```

In a different session, we can change the backend to `tensorflow`.

```
import mrmustard.math as math
math.change_backend("tensorflow")

math.cos(0.1)  # tensorflow
```

Optimization

The `mrmustard.training.Optimizer` uses Adam underneath the hood for the optimization of Euclidean parameters, a custom symplectic optimizer for Gaussian gates and states and a unitary/orthogonal optimizer for interferometers.

We can turn any simulation in Mr Mustard into an optimization by marking which parameters we wish to be trainable. Let's take a simple example: synthesizing a displaced squeezed state.

```
from mrmustard import math
from mrmustard.lab import Dgate, Ggate, Attenuator
from mrmustard.physics import fidelity
from mrmustard.training import Optimizer

math.change_backend("tensorflow")

D = Dgate(x = 0.1, y = -0.5, x_trainable=True)
L = Attenuator(transmissivity=0.5)

# we write a function that takes no argument
def cost_fn_eucl():
    state_out = Vacuum(1) >> D >> L
    return 1 - fidelity(state_out, Coherent(0.5))

G = Ggate(num_modes=1, symplectic_trainable=True)
def cost_fn_symp():
    state_out = Vacuum(1) >> G >> D >> L
    return 1 - fidelity(state_out, Displaced(0.5))

# For illustration, here the Euclidean optimizer
opt = Optimizer(symplectic_lr=0.1, euclidean_lr=0.1)
opt.minimize(cost_fn_eucl, by_optimizing=[D])

# But the symplectic optimization always does
opt = Optimizer(symplectic_lr=0.1, euclidean_lr=0.1)
opt.minimize(cost_fn_symp, by_optimizing=[G])
```



Help

[Installing packages](#) 
[Uploading packages](#) 
[User guide](#) 
[Project name retention](#) 
[FAQs](#)

About PyPI

[PyPI Blog](#) 
[Infrastructure dashboard](#) 
[Statistics](#)
[Logos & trademarks](#)
[Our sponsors](#)

Contributing to PyPI

[Bugs and feedback](#)
[Contribute on GitHub](#) 
[Translate PyPI](#) 
[Sponsor PyPI](#)
[Development credits](#) 

Using PyPI

[Terms of Service](#) 
[Report security issue](#)
[Code of conduct](#) 
[Privacy Notice](#) 
[Acceptable Use Policy](#) 

Status: [All Systems Operational](#) 

Developed and maintained by the Python community, for the Python community.
[Donate today!](#)

"PyPI", "Python Package Index", and the blocks logos are registered [trademarks](#) of the [Python Software Foundation](#) .

© 2026 [Python Software Foundation](#) 
[Site map](#)

Deployed from `72387a4` 

[Switch to desktop version](#)

> [English](#) [español](#) [français](#) [日本語](#) [português \(Brasil\)](#) [українська](#) [Ελληνικά](#) [Deutsch](#) [中文 \(简体\)](#)
[中文 \(繁體\)](#) [русский](#) [עברית](#) [Esperanto](#) [한국어](#)



AWS
Cloud computing
and Security
Sponsor



Datadog
Monitoring



Depot
Continuous
Integration



Fastly
CDN



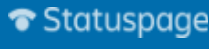
Google
Download Analytics



Pingdom
Monitoring



Sentry
Error logging



StatusPage
Status page